# Parallelism & Concurrency Assignment 3 Report

Neha Chakraborty (373384) Guillaume Marie Lepin (381189)

## 1) Explain how you implemented and optimized rmm_gpu

***a) Identify the different ways of splitting the work among threads and thread blocks, and discuss their advantages and disadvantages***

Several thread and block decomposition strategies were explored to improve arithmetic efficiency, memory reuse, and GPU occupancy while keeping the implementation scalable for large matrices.

The baseline implementation used a simple two-dimensional decomposition where each CUDA thread computed one output element of matrix $C$. Thread blocks were mapped directly onto the output matrix using `blockIdx.x` and `blockIdx.y`, while `threadIdx.x` and `threadIdx.y` identified the thread position inside the block.

- **Advantages:** The implementation was straightforward, easy to debug, and mapped naturally onto the output matrix structure.
- **Disadvantages:** Neighboring threads repeatedly loaded identical values from global memory, producing redundant memory traffic and poor arithmetic intensity. As matrix sizes increased, the kernel became increasingly limited by memory bandwidth.

To reduce redundant computation, later implementations introduced GPU-side reduction kernels. These kernels used a one-dimensional decomposition where each thread computed one element of either `redA` or `redB`. A block size of 256 threads was chosen to maintain good occupancy while preserving balanced work distribution across streaming multiprocessors.

- **Advantages:** The reduction stage executed efficiently in parallel with simple index calculations and uniform workload distribution.
- **Disadvantages:** The implementation required intermediate reduced matrices, introducing additional global memory allocations and extra kernel launches.

For the matrix multiplication stage, a tiled two-dimensional decomposition was adopted. Threads were organized into $16 \times 16$ thread blocks (256 threads total, corresponding to 8 warps). Each thread computed one output element. Sub-tiles were loaded cooperatively into shared memory.

- **Advantages:** Shared-memory tiling allowed threads within the same block to reuse loaded matrix values, reducing repeated global-memory accesses and improving memory locality.
- **Disadvantages:** Larger tile sizes increase shared-memory and register usage per block, which can reduce occupancy if hardware resource limits are exceeded.

The final submitted implementation uses a fused tiled kernel. Instead of storing reduced matrices explicitly in global memory, it computes the row and column reductions dynamically while loading each tile into shared memory.

- **Advantages:** Kernel fusion removes intermediate global-memory writes, avoids extra global-memory allocations, and reduces kernel-launch overhead.
- **Disadvantages:** The indexing logic is more complex and the fused kernel has slightly less tuning flexibility than the separated reduction and multiplication implementation.

***b) List all optimizations you applied and the reasons why you chose to apply them***

The primary optimization was the algebraic reformulation of the RMM equation. The original implementation computed four separate products for every $(i, j, k)$ combination. By reformulating the computation using row and column reductions, the number of required multiplications per output element was reduced while preserving correctness.

The reformulation is algebraically equivalent to the original RMM expression:

$$\sum_{a=0}^{1}\sum_{b=0}^{1} A_{2i+a,k}B_{k,2j+b} = (A_{2i,k}+A_{2i+1,k})(B_{k,2j}+B_{k,2j+1})$$

Moving the reduction stage onto the GPU avoided CPU-side preprocessing bottlenecks and eliminated unnecessary synchronization between host and device.

Shared-memory tiling was applied because neighboring threads reuse overlapping matrix regions during multiplication. Loading tiles into shared memory reduced global-memory bandwidth pressure and improved locality.

A thread block size of $16 \times 16$ was selected because it provides 256 threads per block (8 warps), which is large enough to expose substantial thread-level parallelism while remaining small enough to avoid excessive shared-memory and register pressure. Larger tiles can increase data reuse but may reduce occupancy due to higher resource consumption per block, whereas smaller tiles generally reduce reuse opportunities and increase scheduling overhead. The $16 \times 16$ configuration therefore provides a practical compromise between occupancy, memory reuse, and implementation simplicity.

The final fused implementation combines reduction and multiplication in one kernel. During each tile load, the kernel computes

$$A_{2i,k} + A_{2i+1,k} \quad \text{and} \quad B_{k,2j} + B_{k,2j+1}$$

directly into shared memory and then performs the tiled multiply.

Compiler-level refinements such as `__restrict__` and loop unrolling were included in the optimized variants. They were not benchmarked as standalone optimizations; they are small supporting improvements applied after the main algorithmic changes.

Correctness was verified by comparing the GPU output against the provided single-threaded CPU implementation for small and medium input sizes. The comparison checked that the generated matrix entries matched exactly.

## 2) Report how much each optimization improves performance and explain the result obtained

***Design and run experiments that isolate the effect of each optimization/thread organization you tried out and then report the results***

To isolate optimization effects, several CUDA variants were benchmarked independently. Timings were measured under the same compilation settings and using the same matrix-generation routine. One warm-up iteration was discarded, and the reported runtimes represent averages over three measured executions.

The evaluated implementations are summarized in Table 1, while the corresponding runtimes are shown in Table 2. Table 2 reports the total measured execution time printed by the program for each variant. The phase breakdown in Section 3 uses CUDA-event timings inside `rmm_gpu`. It therefore isolates the host-to-device copy, kernel execution, and device-to-host copy phases, but does not include overhead outside the marked CUDA-event regions.

| Variant | Description |
|---|---|
| baseline | Direct GPU implementation without algebraic reduction |
| impl_1 | CPU-side reduction with GPU multiplication |
| impl_2 | GPU-side reduction with untiled multiplication |
| impl_4 | GPU-side reduction with tiled multiplication |
| impl_3 | Fused reduction and tiled multiplication, submitted final version |

**Table 1:** GPU implementation variants evaluated.

The comparisons were structured to isolate specific optimization strategies:

- **`direct GPU baseline` vs `impl_1`:** Measures the effect of algebraic reformulation.
- **`impl_1` vs `impl_2`:** Measures the impact of moving reductions from CPU preprocessing onto the GPU.
- **`impl_2` vs `impl_4`:** Measures the effect of shared-memory tiling.
- **`impl_4` vs `impl_3`:** Measures the effect of kernel fusion.

At small matrix sizes, execution time is dominated by fixed CUDA overheads, synchronization, and allocation costs. Consequently, runtime differences between implementations remain small until larger problem sizes are reached. The most meaningful comparisons therefore occur at $N = 4096$.

Moving from the direct GPU baseline to `impl_1` reduces runtime from 0.1933 s to 0.1393 s, corresponding to a **1.39× speedup**. This improvement is primarily due to the algebraic reduction, which decreases the number of arithmetic operations required per output element.

Moving reductions onto the GPU in `impl_2` further reduces runtime to 0.1316 s, producing an additional **1.06× speedup** by removing CPU preprocessing overhead.

Adding shared-memory tiling in `impl_4` reduces runtime from 0.1316 s to 0.1253 s, corresponding to a further **1.05× speedup**. This improvement comes from reducing repeated global-memory accesses during multiplication. Once a tile is loaded into shared memory, multiple threads can reuse the same data, increasing arithmetic intensity and lowering pressure on global-memory bandwidth.

The fused implementation `impl_3` reaches 0.1215 s, producing a further **1.03× speedup** relative to `impl_4` by eliminating intermediate global-memory writes and reducing kernel-launch overhead. Since this was the fastest measured variant, it was used as the final submitted implementation.

The experiments isolate the main algorithmic and thread-organization changes. Smaller compiler-level choices such as `__restrict__`, loop unrolling, and coalesced indexing were kept in the optimized variants and were not benchmarked as separate variants.

Overall, the final implementation reduces runtime from 0.1933 s to 0.1215 s at $N = 4096$, corresponding to an overall **1.59× speedup** relative to the direct GPU baseline. The results indicate that the algebraic reformulation provides the largest performance improvement. This is expected because it reduces the amount of arithmetic work performed by the kernel before any hardware-specific optimizations are applied. In contrast, tiling and kernel fusion primarily improve memory efficiency and reduce overheads, resulting in smaller but still measurable gains. This highlights the importance of algorithmic optimizations, which often provide larger benefits than low-level implementation refinements alone.

| Matrix size | baseline (s) | impl_1 (s) | impl_2 (s) | impl_4 (s) | impl_3 / final (s) |
|---|---|---|---|---|---|
| 16 | 0.0832 | 0.0774 | 0.0777 | 0.0760 | 0.0787 |
| 64 | 0.0772 | 0.0772 | 0.0760 | 0.0759 | 0.0789 |
| 256 | 0.0778 | 0.0787 | 0.0766 | 0.0763 | 0.0789 |
| 1024 | 0.0818 | 0.0817 | 0.0826 | 0.0817 | 0.0805 |
| 4096 | 0.1933 | 0.1393 | 0.1316 | 0.1253 | 0.1215 |

**Table 2:** Total execution runtimes across the GPU variants. The optimization effects become visible primarily at $N = 4096$, where fixed overheads no longer dominate runtime.

## 3) For each of the following $\langle M, N, K \rangle$ combinations

***a) In a table, present the execution times you measured for running the baseline CPU version and your optimized GPU version***

The baseline single-threaded CPU implementation was benchmarked against the optimized GPU implementation across dimensions ranging from $N = 16$ to $N = 4096$, where $M = N = K$.

GPU timings in Table 3 report the active CUDA phases measured inside `rmm_gpu`: host-to-device transfers, kernel execution, and device-to-host transfers. CPU runtimes correspond to single executions, while GPU values represent averages over three measured runs. These GPU values are lower than the full program-level runtimes in Table 2, because Table 2 includes additional overhead outside the CUDA-event regions. The distinction is intentional: Table 3 and Table 4 use the same GPU timing basis, so the CPU/GPU comparison and phase breakdown are directly comparable.

| $M{=}N{=}K$ | CPU (s) | GPU (s) | Speedup |
|---|---|---|---|
| 16 | $3.00 \times 10^{-6}$ | $4.93 \times 10^{-4}$ | 0.006× |
| 64 | $1.65 \times 10^{-4}$ | $4.66 \times 10^{-4}$ | 0.35× |
| 256 | $1.11 \times 10^{-2}$ | $6.35 \times 10^{-4}$ | 17.4× |
| 1024 | 3.375 | $2.94 \times 10^{-3}$ | 1,146× |
| 4096 | 605.5 | $4.32 \times 10^{-2}$ | 14,006× |

**Table 3:** CPU baseline versus optimized GPU implementation. GPU times are CUDA-event measurements of the active H→D, compute, and D→H phases.

At small matrix sizes, the CPU outperforms the GPU because the computation is too small to amortize CUDA launch and transfer overheads. The GPU begins outperforming the CPU at $N = 256$, and the speedup increases substantially for larger matrices. At $N = 4096$, the optimized GPU implementation achieves a speedup of approximately 14,000× relative to the sequential CPU baseline for the active GPU phases.

***b) Present graphically the breakdown of the GPU runtime for copying data from host to device, computation and copying data from device to host***

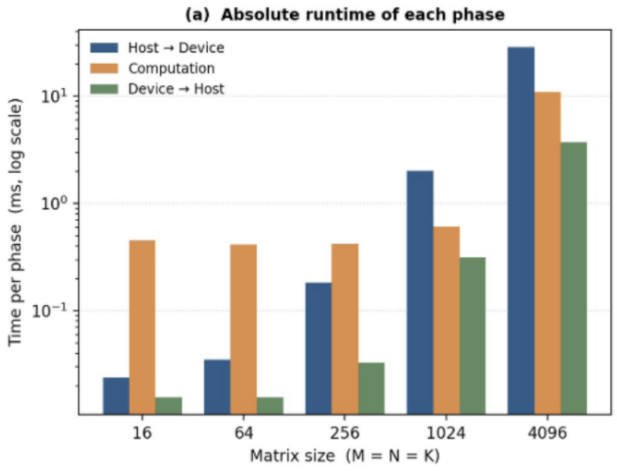
To analyze GPU runtime distribution, the execution was divided into three phases:

1. Host-to-device memory transfers
2. GPU kernel computation
3. Device-to-host memory transfers

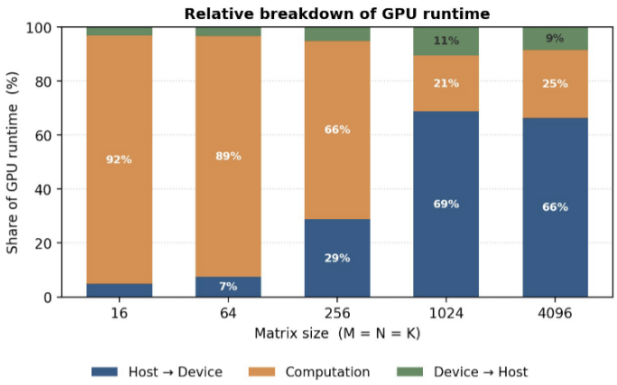
Each phase was timed independently using CUDA events. The runtime breakdown is shown graphically in Figure 1 and numerically in Table 4. At small matrix sizes, fixed CUDA overheads dominate runtime measurements. As matrix dimensions increase, memory-transfer costs scale more rapidly. At $N = 4096$, the host-to-device transfer requires 28.643 ms, computation requires 10.870 ms, and the device-to-host transfer requires 3.720 ms.

| $M{=}N{=}K$ | H→D | Computation | D→H | Total |
|---|---|---|---|---|
| 16 | 0.024 | 0.453 | 0.016 | 0.493 |
| 64 | 0.035 | 0.416 | 0.016 | 0.466 |
| 256 | 0.182 | 0.419 | 0.033 | 0.635 |
| 1024 | 2.025 | 0.608 | 0.312 | 2.945 |
| 4096 | 28.643 | 10.870 | 3.720 | 43.233 |

**Table 4:** Per-phase GPU runtime breakdown in milliseconds, corresponding to Figure 1.




(a) Absolute runtime of each phase (log scale).




(b) Relative breakdown of GPU runtime.

**Figure 1:** Per-phase GPU runtime breakdown. (a) Absolute timings show that host-to-device transfers become the dominant cost for large matrices. (b) Relative runtime distribution across execution phases.

## 4) Analyze and explain your results from the previous question

The experimental results demonstrate that architectural differences between the CPU and GPU dictate their respective performance envelopes across varying problem sizes. At ultra-small workloads ($N = 16$), the sequential CPU implementation is faster because the arithmetic workload is too small to mask fixed overheads, including CUDA kernel-launch latency, memory-transfer setup, and device synchronization.

The structural performance crossover occurs between $N = 64$ and $N = 256$. Beyond this scale, the problem size yields enough parallel work to use the GPU's streaming multiprocessors effectively and hide instruction latency. As matrix dimensions scale up to $N = 4096$, the GPU executes the active RMM phases in roughly 43.2 ms, while the single-threaded CPU baseline requires more than 600 seconds. The increasing speedup with matrix size reflects the fact that GPU utilization improves as more independent work becomes available. For small matrices, many GPU resources remain underutilized and fixed launch overheads dominate execution time. As the workload grows, these fixed costs are amortized over a much larger number of arithmetic operations, allowing the GPU to better exploit its massive parallelism.

A critical observation from the runtime breakdown is the shifting bottleneck from compute-bound behavior to communication cost. At $N = 4096$, the host-to-device memory transfer demands 28.643 ms, representing about 66.25% of the timed GPU duration. The input data volume scales quadratically with matrix dimension: moving from $N = 1024$ to $N = 4096$ increases the input size by $16\times$. The measured H→D time increases from 2.025 ms to 28.643 ms, which is larger than the ideal scaling factor and likely reflects fixed overheads, bandwidth saturation effects, and measurement variability.

Conversely, the device-to-host transfer is smaller because the output matrix dimensions are reduced to $(M/2) \times (K/2)$. The returned matrix therefore has only 25% of the elements of a full $M \times K$ matrix. At $N = 4096$, the device-to-host transfer time of 3.720 ms is consistent with this smaller output size.

These dynamics show that, for the measured GPU phases including transfers, host-to-device communication is the dominant cost at large matrix sizes. The kernel itself is much faster than the CPU baseline, but the measured GPU runtime is increasingly limited by data movement. To mitigate this bottleneck in a production implementation, standard pageable host memory could be replaced with pinned memory via `cudaHostAlloc()`, and larger inputs could be processed using asynchronous chunks with CUDA streams so that transfers and computation overlap.

Finally, the gap between the total program-level measurements and the CUDA-event phase measurements comes from overhead outside the marked H→D, compute, and D→H regions. This includes costs such as CUDA setup, allocation overhead, host-side matrix initialization, and final CSV serialization. Separating full program-level timings from CUDA-event phase timings provides a clearer view of both end-to-end behavior and kernel-level efficiency.